2026-7-22 Linux文件--ELF文件系统

- 1. 从普通文件到 ELF 文件
  - 1.1 文件系统先解决“字节放在哪里”
  - 1.2 ELF 解决“这些字节如何解释”
  - 1.3 从源文件到 ELF 的主线
  - 1.4 为什么需要理解 ELF
- 2. 编译与目标文件
  - 2.1 从源代码到可重定位目标文件
    - 2.1.1 预处理
    - 2.1.2 编译
    - 2.1.3 汇编与链接
  - 2.2 编译过程中“节”逐步形成
- 3. ELF 的四种典型文件
  - 3.1 可重定位文件
  - 3.2 可执行文件
  - 3.3 共享目标文件
  - 3.4 内核转储文件
- 4. ELF 文件的整体结构
  - 4.1 ELF 的四部分
  - 4.2 ELF Header
  - 4.3 Program Header Table
  - 4.4 Section Header Table
  - 4.5 Sections
    - 4.5.1 代码节 .text
    - 4.5.2 已初始化数据 .data
    - 4.5.3 未初始化数据 .bss
    - 4.5.4 只读数据 .rodata
    - 4.5.5 符号表与字符串表
    - 4.5.6 重定位节
- 5. Section 与 Segment
  - 5.1 为什么同时存在节和段
  - 5.2 从节到段的合并
  - 5.3 合并减少页面浪费
  - 5.4 合并便于设置权限
- 6. 链接：把多个 ELF 目标文件组合起来
  - 6.1 链接器需要解决的两类问题

        - 6.1.1 符号解析
        - 6.1.2 重定位
    - 6.2 合并同名节
    - 6.3 静态链接
    - 6.4 动态链接
- 7. ELF 如何变成进程
    - 7.1 execve 替换进程映像
    - 7.2 内核首先读取 ELF 头
    - 7.3 映射 PT_LOAD 段
    - 7.4 程序在加载前已经有虚拟地址
    - 7.5 动态加载器
    - 7.6 程序入口不是 main
    - 7.7 加载后的用户地址空间
- 8. 用工具观察 ELF
    - 8.1 file
    - 8.2 readelf -h
    - 8.3 readelf -l
    - 8.4 readelf -S
    - 8.5 objdump
    - 8.6 ldd
- 9. 一个完整例子
    - 9.1 源文件
    - 9.2 生成目标文件
    - 9.3 链接
    - 9.4 检查并运行
- 10. 核心结论
    - 10.1 文件格式与文件系统是两层问题
    - 10.2 节面向链接，段面向加载
    - 10.3 .o、.so 和可执行文件都属于 ELF
    - 10.4 main 不是最先进去的地方
    - 10.5 编译链接与加载是一条连续链

1. 从普通文件到 ELF 文件

1.1 文件系统先解决“字节放在哪里”

磁盘、分区、文件系统和 inode 解决的是文件的定位问题：

```
路径
  → 目录项
  → inode
  → 数据块
  → 文件内容
```

对一个普通文件而言，文件系统主要负责保存：

- 文件属性；
- 文件内容所占的数据块；
- 文件名到 inode 号的目录映射；
- 文件大小、权限和读写位置等管理信息。

从文件系统角度看，`.c`、`.o`、`.so` 和可执行文件都只是普通文件。真正区分它们的，是文件内容的格式以及文件权限。

1.2 ELF 解决“这些字节如何解释”

二进制文件不是把机器码和数据的字节随意拼在一起。Linux 中常见的可重定位目标文件、可执行文件和共享库都遵循 ELF 格式，即：

> ELF = Executable and Linkable Format，可执行与可链接格式

ELF 为文件内容规定了统一结构，使链接器、加载器和调试器能够回答：

- 哪一部分是指令；
- 哪一部分是已初始化数据；
- 哪一部分是未初始化数据；
- 程序入口在哪里；
- 哪些内容需要加载到内存；
- 加载后的虚拟地址和访问权限是什么；
- 哪些符号需要链接或重定位；
- 依赖哪些共享库；
- 是否包含调试信息。

因此可以把两者的分工概括为：

> 文件系统负责找到 ELF 文件的字节，ELF 格式负责解释这些字节。

1.3 从源文件到 ELF 的主线

一个 C/C++ 程序通常经历：

```
源文件 .c/.cpp
  → 预处理
  → 编译
  → 汇编
  → 可重定位目标文件 .o
  → 链接
  → 可执行文件或共享目标文件 .so
  → 启动时加载
  → 进程执行
```

这条主线正是本篇要展开的内容。

1.4 为什么需要理解 ELF

IDE 往往把编译和链接封装成“一键构建”，程序能运行时不容易注意到内部格式。但遇到链接错误时，问题可能来自：

- 找不到符号定义；
- 符号重复定义；
- 目标文件类型或架构不匹配；
- 静态库与动态库选择错误；
- 共享库路径或依赖关系错误；
- 需要重定位的位置没有正确处理。

理解 ELF 之后，`readelf`、`objdump`、`file`、`ldd` 等工具给出的信息会从“难懂的输出”变成可追踪的结构。

2. 编译与目标文件

2.1 从源代码到可重定位目标文件

假设一个程序由两个源文件组成：

```
hello.c  → 调用 code.c 中定义的 run()
```

```
code.c   → 定义 run()
```

可以分别编译：

```
gcc -c hello.c
gcc -c code.c
```

得到：

```
hello.o
code.o
```

它们是可重定位目标文件。重新修改一个源文件时，只需重新编译对应目标文件，不必无条件重新编译整个工程。

2.1.1 预处理

预处理器主要处理：

- `#include`，把头文件内容展开；
- `#define`，进行宏替换；
- 条件编译；
- 行号、源文件名等调试信息。

可以把结果单独输出，便于观察：

```
gcc -E hello.c -o hello.i
```

2.1.2 编译

编译器把预处理后的 C/C++ 代码转换为汇编指令：

```
gcc -S hello.i -o hello.s
```

这一阶段会进行语法和语义分析、优化，并生成与目标处理器指令集相关的汇编代码。

2.1.3 汇编与链接

汇编器把汇编代码转换为机器指令和数据，生成 `.o`：

```
gcc -c hello.s -o hello.o
```

`.o` 已经是二进制文件，但通常不能直接运行，因为它可能仍包含：

- 未解析的外部符号；
- 需要链接器修改的地址；
- 尚未合并的节；
- 对其他目标文件或库的依赖。

最后由链接器生成可执行文件：

```
gcc hello.o code.o -o hello
```

2.2 编译过程中“节”逐步形成

`.o` 文件不是一团没有边界的字节，而是被划分为多个节：

```
.o
├─ .text        机器指令
├─ .data        已初始化全局变量、静态变量
├─ .bss         未初始化或零初始化数据所需的空间
├─ .rodata      只读数据
├─ .symtab      符号表
├─ .strtab      字符串表
└─ .rel.*       重定位信息
```

这些节具有不同用途和访问属性，为后续链接与加载提供依据。

3. ELF 的四种典型文件

3.1 可重定位文件

典型文件：

```
xxx.o
```

可重定位文件包含准备好与其他目标文件链接的代码和数据。它保存尚未最终确定的符号引用与重定位信息，通常不能独立作为普通程序运行。

常见 ELF 类型：

```
ET_REL
```

3.2 可执行文件

典型文件：

```
a.out
```

可执行文件包含程序入口、加载信息及已经完成链接的内容。传统非 PIE 可执行文件常见类型为：

```
ET_EXEC
```

现代 Linux 中很多可执行文件采用位置无关可执行文件 PIE，类型表现为：

```
ET_DYN
```

不能只根据类型名判断“这是可执行程序”还是“共享库”，还要结合 ELF 标志、入口、解释器和运行方式分析。

3.3 共享目标文件

典型文件：

```
xxx.so
```

共享目标文件可以在程序启动或运行期间被加载，由动态链接器/加载器建立地址关系。它通常也是 `ET_DYN`，便于被映射到不同进程的地址空间。

3.4 内核转储文件

典型文件：

```
core
```

进程异常终止时可生成 core dump，其中保存进程的地址空间、寄存器等执行上下文，常用于事后调试。

常见 ELF 类型：

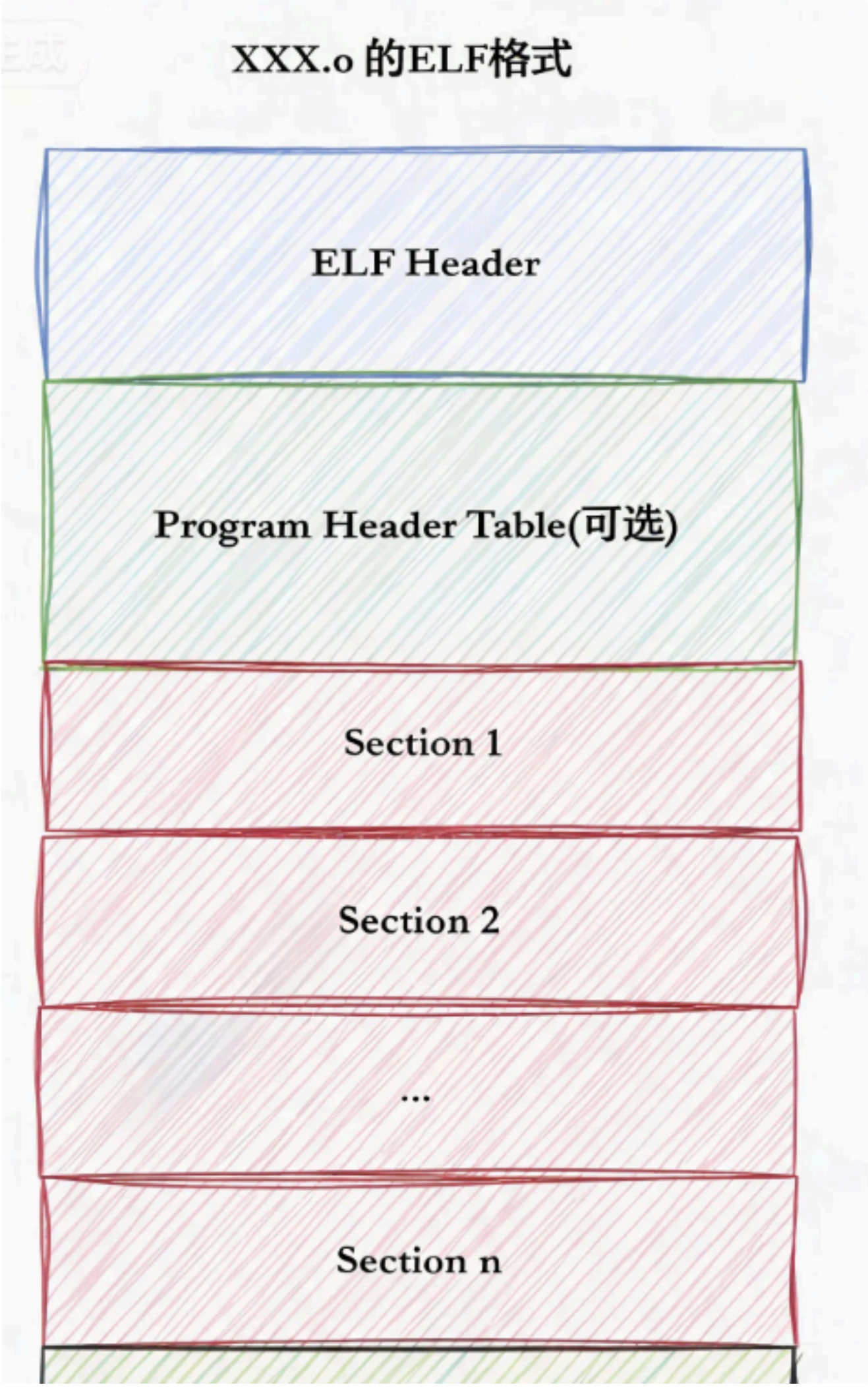
```
ET_CORE
```

4. ELF 文件的整体结构

4.1 ELF 的四部分

课堂用四个基本部分描述 ELF：

```
ELF Header
Program Header Table
Sections
Section Header Table
```


XXX.o 的ELF格式
ELF Header
Program Header Table(可选)
Section 1
Section 2
...
Section n



这些部分不一定要求物理上严格按某个连续顺序排列，但 ELF 头会记录各表和内容的文件偏移、大小及数量。

4.2 ELF Header

ELF 头位于文件开头，是解析整个文件的入口。它主要描述：

- ELF 魔数和版本；
- 32 位或 64 位；
- 小端或大端；
- 目标体系结构；
- 文件类型；
- 程序入口地址；
- 程序头表的位置、大小和数量；
- 节头表的位置、大小和数量；
- ELF 头自身大小；
- 若干标志信息。

ELF 文件的起始魔数是：

```
7f 45 4c 46
```

对应的 ASCII 字符是：

```
\x7f E L F
```

加载器首先检查魔数、文件类型、机器架构等信息，判断文件是否可以按当前平台要求执行。

4.3 Program Header Table

程序头表描述加载视图，每个程序头通常对应一个段 `Segment`。程序头会说明：

- 段的类型；
- 段在文件中的偏移；
- 段的文件大小；
- 加载到内存后的虚拟地址；
- 所需内存大小；
- 对齐要求；
- 读、写、执行权限。

常见段类型包括：

- `PT_LOAD`：需要映射到进程地址空间的内容；
- `PT_INTERP`：指定动态加载器路径；
- `PT_DYNAMIC`：动态链接信息；
- `PT_PHDR`：程序头表自身；
- `PT_NOTE`：附加说明信息；
- `PT_TLS`：线程局部存储。

4.4 Section Header Table

节头表描述链接与静态分析视图。每个节头会记录：

- 节名称；
- 节类型；
- 节在文件中的偏移；
- 节大小；
- 对齐；
- 标志与关联信息。

节头表对链接器、反汇编器和调试器很重要；程序运行时不一定需要依靠节头表定位每一段代码。

4.5 Sections

节是 ELF 组织内容的基本单位。一个节通常只承担一种主要用途，例如代码、只读数据、符号表或重定位信息。

4.5.1 代码节 `.text`

`.text` 保存 CPU 执行的机器指令，是程序的主体代码。

对代码节通常设置：

```
读权限：需要
写权限：通常没有
执行权限：需要
```

只读可执行有助于防止程序在运行时意外改写指令。

4.5.2 已初始化数据 `.data`

`.data` 保存初始化值不为零的全局变量和静态变量。

例如：

```
int global_value = 42;
```

变量本身的存储位置由具体程序决定，但通常会落到可写的数据节或相关段中。

4.5.3 未初始化数据 `.bss`

`.bss` 用于描述未初始化或零初始化的全局变量和静态变量：

```
int global_buffer[1024];
```

这类数据不需要把大量全零字节完整保存在磁盘文件中。ELF 记录它需要占用的内存大小，加载时再为对应区域建立零页或零填充。

因此：

```
文件大小较小
≠
进程运行时占用的虚拟内存较小
```

`.bss` 占用运行时地址空间，但通常不会在文件中占据同样大小的物理字节。

4.5.4 只读数据 `.rodata`

字符串常量和只读常量通常进入 `.rodata`。该节一般可读、不可写，也不作为普通代码执行。

4.5.5 符号表与字符串表

`.symtab` 保存符号信息，例如函数名、全局变量名、符号所在节和值。

`.strtab`、`.dynstr` 等字符串表保存符号名、依赖库名等字符串，供符号表和动态链接结构引用。

没有符号表时，代码仍可能执行，但链接、调试和符号定位会更困难。

4.5.6 重定位节

`.rel.text`、`.rela.text`、`.rela.dyn` 等节保存重定位信息。

重定位解决的是：

> 编译某个目标文件时还不知道最终地址，怎样才能在链接或加载时把正确地址填进指令或数据中？

常见来源包括：

- 调用其他目标文件中的函数；
- 访问外部全局变量；
- 引用共享库符号；
- 位置无关代码需要的地址修正。

5. Section 与 Segment

5.1 为什么同时存在节和段

节 `Section` 主要服务于：

- 编译与链接；
- 符号解析；
- 重定位；
- 反汇编；
- 调试；
- 文件内容分类。

段 `Segment` 主要服务于：

- 可执行文件的加载；
- 虚拟内存映射；
- 页面权限设置；
- 运行时访问控制。

可以用一句话概括：

```
Section：链接器看到的细小分类
Segment：加载器看到的内存映射单元
```

5.2 从节到段的合并

加载一个 ELF 程序时，不会把每个节都单独映射成一个拥有独立权限的内存区域，而是把多个具有相同或兼容属性的节合并为段。

例如：

```
只读代码类节
  .init
  .plt
  .text
  .fini
      ↓ 合并
可读、可执行的 LOAD 段
```

```
可写数据类节
  .data
  .got
  .bss
```

```
        ↓ 合并
可读、可写的 LOAD 段
```

具体合并关系由链接器确定，并记录在 ELF 的程序头表中。

5.3 合并减少页面浪费

内存映射和权限管理通常按页进行，常见页大小为 `4096 B`。

若多个节分别映射，即使一个节很小，也可能分别占用至少一个页。把相邻、同属性节组合到同一个段，可以减少：

- 页对齐造成的空洞；
- 页表项数量；
- 内存映射次数；
- 碎片和管理开销。

5.4 合并便于设置权限

同一个段应尽量拥有一致的权限。

例如代码段采用“可读 + 可执行”，数据段采用“可读 + 可写但不执行”。这样既能正常运行，又比把整个文件映射成可读、可写、可执行更安全。

> 段合并同时服务于空间利用和权限隔离。

6. 链接：把多个 ELF 目标文件组合起来

6.1 链接器需要解决的两类问题

6.1.1 符号解析

源文件之间可能互相调用函数或访问变量，但每个 `.o` 是分别生成的。

例如：

```
hello.o：引用 run()
code.o ：定义 run()
```

链接器需要建立“引用 → 定义”的关系。若只有引用而没有定义，就可能出现未定义符号错误。

如果多个目标文件提供了同名的强符号定义，又可能产生重复定义错误。

6.1.2 重定位

仅有符号对应关系还不够。指令中需要填入最终地址，但 `.o` 生成时并不知道：

- 最终可执行文件从哪里加载；
- 目标函数位于哪个虚拟地址；
- 共享库在运行时会被映射到哪里。

链接器根据重定位记录修改相关位置，使程序入口、函数调用和数据访问能够指向正确地址。

6.2 合并同名节

链接器会把多个目标文件中的同名或同类节组合起来：

```
hello.o.text + code.o.text
  → 可执行文件中的代码区域

hello.o.data + code.o.data
  → 可执行文件中的可写数据区域
```

这一步不是简单拼接字节，还涉及对齐、符号地址计算、重定位和库合并。

6.3 静态链接

静态链接在程序生成阶段把所需库代码复制或合并进可执行文件：

```
目标文件 .o
  + 静态库 .a 中被引用的目标文件
  → 可执行文件
```

静态库 `libxxx.a` 通常是对多个 `.o` 的归档，链接器从中抽取实际需要的成员。

优点：

- 运行时通常不依赖对应 `.so`；
- 部署时需要的动态库少；
- 调用路径明确。

缺点：

- 可执行文件更大；
- 多个程序重复包含相同库代码；
- 库更新后往往需要重新链接。

可用 `-static` 要求尽可能静态链接：

```
gcc hello.c -static -o hello_static
```

目标系统中必须实际存在可用的静态库。

6.4 动态链接

动态链接不把共享库的全部内容直接复制进可执行文件，而是在 ELF 中记录依赖关系：

```
可执行文件
  → 依赖 libfoo.so
  → 动态加载器在启动或运行时定位并映射共享库
```

优点：

- 多个程序可以共享同一份库代码；
- 可执行文件通常更小；
- 修复共享库后可被多个程序复用。

代价：

- 运行时必须找到兼容的共享库；

- 动态加载和重定位需要额外工作；
- 库路径或 ABI 不兼容会导致启动失败。

常见动态链接依赖信息位于 `.dynamic`、`.dynsym`、`.dynstr`、`.rela.dyn`、`.rela.plt` 等结构中。

7. ELF 如何变成进程

7.1 execve 替换进程映像

Shell 执行命令时，通常会调用 `fork()` 创建子进程，再由子进程调用 `execve()` 加载目标程序。

`execve()` 成功后会替换当前进程的用户空间映像，包括代码、数据、堆、栈等，但保留进程身份和已经安排的某些资源。

概念流程：

```
shell
  → fork()
  → 子进程 execve("./hello", argv, envp)
  → 内核读取 ELF
  → 建立新的用户地址空间映射
  → 跳转到 ELF 入口
```

7.2 内核首先读取 ELF 头

内核检查目标文件是否具有合法格式，例如：

- 文件是否具有执行权限；
- ELF 魔数是否正确；
- 32/64 位和字节序是否受支持；
- 目标机器架构是否匹配；
- 文件类型是否适合执行；
- 程序头表和入口是否有效。

不合法的二进制即使有执行权限，也不能被正常作为程序加载。

7.3 映射 PT_LOAD 段

加载器根据程序头表中的 `PT_LOAD` 项，把文件内容映射到进程的虚拟地址空间。

程序头同时描述：

```
文件中从哪里读取
读取多少字节
映射到哪个虚拟地址
运行时需要多大空间
页面按什么方式对齐
是否可读、可写、可执行
```

文件内容和内存虚拟地址之间不是简单复制关系，而是通过页映射建立联系。尚未修改的只读代码页可以由多个进程共享。

7.4 程序在加载前已经有虚拟地址

ELF 中的代码、数据、入口和段位置在磁盘文件里就以虚拟地址形式描述。程序尚未获得物理内存页时，仍然拥有虚拟地址布局。

现代 PIE 可执行文件常以相对或可重定位形式构建。加载时会选择一个基地址，再把地址加上对应偏移：

```
运行时地址 = 加载基址 + ELF 中的相对地址
```

这使多个程序可以映射到不同地址，也为地址空间随机化提供基础。

7.5 动态加载器

对于动态链接程序，`PT_INTERP` 段指定动态加载器路径，例如：

```
/lib64/ld-linux-x86-64.so.2
```

动态加载器会：

1. 读取可执行文件的动态链接信息；

2. 找到依赖的共享库；
3. 把共享库映射到地址空间；
4. 递归加载库的依赖；
5. 解析外部符号；
6. 完成必要的重定位；
7. 把控制权交给程序入口。

如果依赖库缺失、架构不匹配或版本不兼容，程序可能在 `main()` 执行前就失败。

7.6 程序入口不是 main

ELF 头中的 `Entry point address` 通常指向 `_start`，而不是用户编写的 `main()`。

简化后的调用关系为：

```
_start
  → C 运行时初始化
  → __libc_start_main
  → main
  → return/exit
```

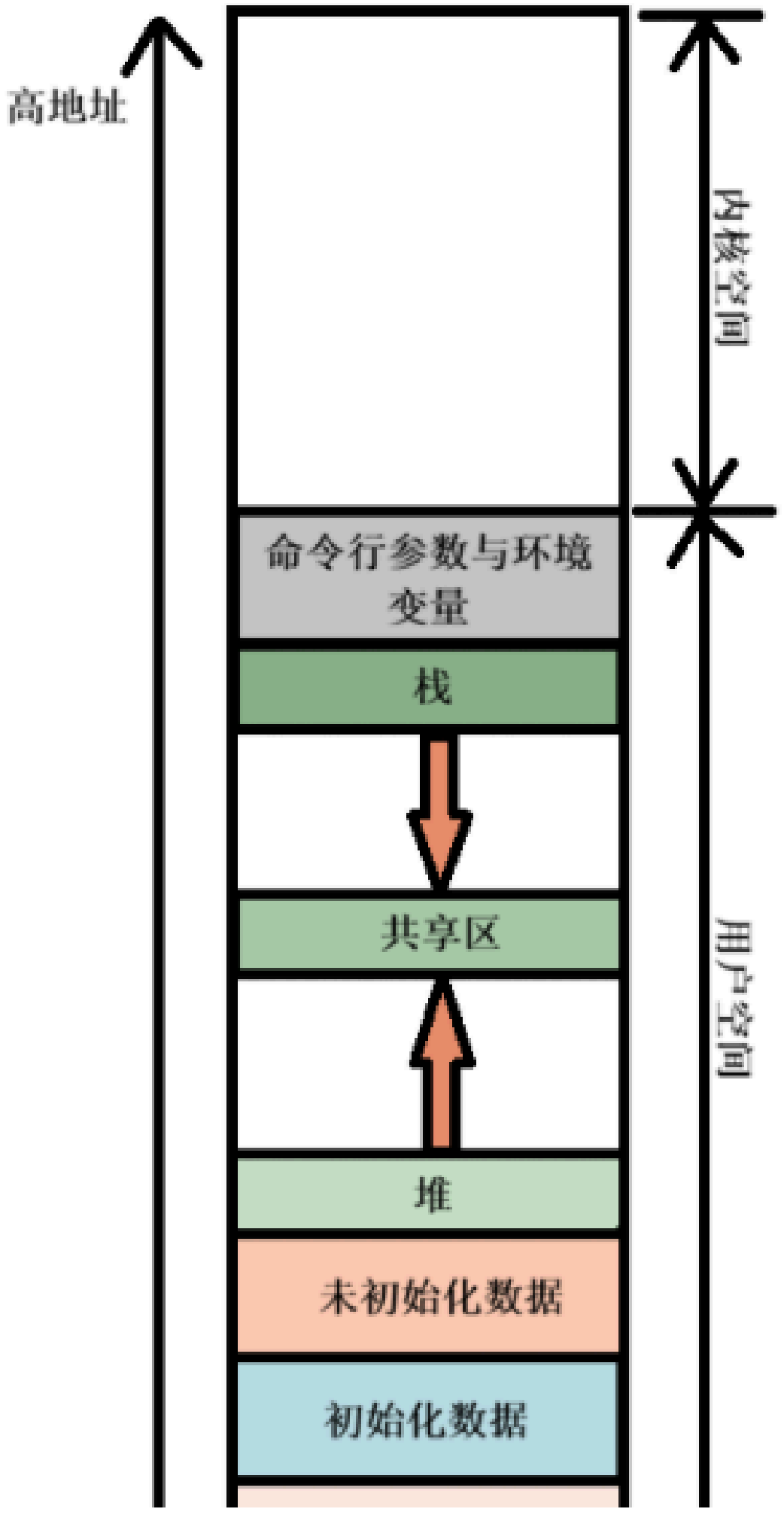
C 运行时初始化可能处理：

- 构造全局对象；
- 初始化线程局部存储；
- 准备环境；
- 注册退出处理；
- 根据 `argc`、`argv`、`envp` 调用 `main()`。

因此：

```
加载器进入的入口 ≠ 用户代码中的 main
```

7.7 加载后的用户地址空间


高地址
内核空间
用户空间
命令行参数与环境变量
栈
共享区
堆
未初始化数据
初始化数据



进程运行时的典型用户空间从低地址向高地址大致包含：

```
代码
只读数据
已初始化数据
未初始化数据
堆
共享库等映射区
栈
命令行参数与环境变量
```

这就是 ELF 中的节和段经过链接、映射与运行时分配之后，在进程地址空间中的体现。

8. 用工具观察 ELF

8.1 file

```
file ./hello
```

可以快速判断文件是否是 ELF、架构、字节序、是否动态链接以及是否带调试信息。

8.2 readelf -h

```
readelf -h ./hello
```

查看 ELF 头，包括：

- Magic；
- Class；
- Data；
- Type；
- Machine；
- Entry point；

- Program/Section Header 的偏移、大小和数量。

```
ZL@VM-0-14-ubuntu:~/code/lesson20-lib$ readelf -h a.out
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x1060
  Start of program headers:          64 (bytes into file)
  Start of section headers:          13976 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         13
  Size of section headers:           64 (bytes)
  Number of section headers:         31
  Section header string table index: 30
```

示例中的 `Type` 为 `DYN (Position-Independent Executable file)`，表示它是现代 PIE 可执行文件，而不是传统的 `ET_EXEC`。

8.3 readelf -l

```
readelf -l ./hello
```

显示程序头表和段映射信息。分析加载过程时，应重点查看：

- `LOAD` 段的文件偏移；
- 虚拟地址；
- 文件大小与内存大小；
- 段权限；
- `INTERP` 与 `DYNAMIC`。

8.4 readelf -S

```
readelf -S ./hello
```

显示节头表，包括节名、类型、地址、偏移、大小、标志和对齐。

```
ZL@VM-0-14-ubuntu:~/code/lesson20-lib$ readelf -S a.out
There are 31 section headers, starting at offset 0x3698:

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  [ 0]                   NULL             0000000000000000  00000000
       0000000000000000  0000000000000000           0     0     0
  [ 1] .interp           PROGBITS         0000000000000318  00000318
       000000000000001c  0000000000000000   A       0     0     1
  [ 2] .note.gnu.pr[...] NOTE             0000000000000338  00000338
       0000000000000030  0000000000000000   A       0     0     8
  [ 3] .note.gnu.bu[...] NOTE             0000000000000368  00000368
       0000000000000024  0000000000000000   A       0     0     4
  [ 4] .note.ABI-tag     NOTE             000000000000038c  0000038c
       0000000000000020  0000000000000000   A       0     0     4
  [ 5] .gnu.hash         GNU_HASH         00000000000003b0  000003b0
       0000000000000024  0000000000000000   A       6     0     8
  [ 6] .dynsym           DYNSYM           00000000000003d8  000003d8
       00000000000000a8  0000000000000018   A       7     1     8
  [ 7] .dynstr           STRTAB           0000000000000480  00000480
       000000000000008f  0000000000000000   A       0     0     1
  [ 8] .gnu.version      VERSYM           0000000000000510  00000510
       000000000000000e  0000000000000002   A       6     0     2
  [ 9] .gnu.version_r    VERNEED          0000000000000520  00000520
       0000000000000030  0000000000000000   A       7     1     8
  [10] .rela.dyn         RELA             0000000000000550  00000550
       00000000000000c0  0000000000000018   A       6     0     8
  [11] .rela.plt         RELA             0000000000000610  00000610
       0000000000000018  0000000000000018  AI       6    24     8
  [12] .init             PROGBITS         0000000000001000  00001000
       000000000000001b  0000000000000000  AX       0     0     4
```

查看节时，应优先关注：

- `.text`：代码；
- `.rodata`：只读数据；
- `.data`：已初始化可写数据；
- `.bss`：运行时零初始化区域；
- `.dynsym`、`.dynstr`：动态符号与字符串；
- `.rela.*`：重定位；
- `.interp`：动态加载器路径。

8.5 objdump

```
objdump -h ./hello
objdump -d ./hello
objdump -S ./hello
```

可以分别查看节头、反汇编和带源码的反汇编。`-S` 是否显示源码取决于可执行文件是否保留调试信息。

8.6 ldd

```
ldd ./hello
```

可观察动态链接依赖和解析结果。

需要注意：对不可信程序执行 `ldd`，某些实现可能触发动态加载过程；安全分析时更适合使用 `readelf -d` 等静态检查方式。

9. 一个完整例子

9.1 源文件

```c
/* code.c */
int value = 42;

int add(int a, int b)
{
    return a + b + value;
}
```

```c
/* hello.c */
#include <stdio.h>

int add(int a, int b);

int main(void)
{
    printf("result = %d\n", add(1, 2));
    return 0;
}
```

9.2 生成目标文件

```
gcc -c code.c -o code.o
```

```
gcc -c hello.c -o hello.o
```

两个 `.o` 都是 ELF 可重定位文件。`hello.o` 引用了 `add` 和 `printf`，但自身并不定义它们。

9.3 链接

```
gcc hello.o code.o -o hello
```

链接器：

1. 解析 `hello.o` 对 `add` 的引用；
2. 在 `code.o` 中找到 `add` 定义；
3. 解析 `printf` 对 C 库的依赖；
4. 合并相关节；
5. 完成重定位；
6. 生成程序头表和运行所需结构。

9.4 检查并运行

```
file hello
readelf -h hello
readelf -l hello
readelf -S hello
ldd hello
./hello
```

从“文件存在”到“程序运行”的完整层次为：

```
inode 与数据块保存 hello 的字节
  → ELF 头说明文件类型与入口
  → 程序头表说明怎样映射段
  → 加载器建立虚拟地址空间
  → 动态加载器处理共享库和重定位
  → CPU 从入口开始执行
```

10. 核心结论

10.1 文件格式与文件系统是两层问题

```
文件系统：路径、inode、数据块、读写
ELF 格式：头、节、段、符号、重定位、入口
```

文件系统负责把 `hello` 的字节稳定地读出来；ELF 解析器与加载器负责把这些字节变成可执行的地址空间。

10.2 节面向链接，段面向加载

- Section 是链接器、工具和调试器关注的细分结构。
- Segment 是加载器和虚拟内存系统关注的内存映射单元。
- 同属性节在链接时被组织到段中，以减少页面浪费并统一权限。

10.3 `.o`、`.so` 和可执行文件都属于 ELF

它们共享同一套文件格式框架，但文件类型、入口、程序头、符号和动态链接信息不同。

10.4 main 不是最先进去的地方

ELF 入口通常指向 `_start`，由 C 运行时完成初始化后再调用 `main()`。

10.5 编译链接与加载是一条连续链

```
源代码
  → 编译成带节和重定位信息的 .o
  → 链接成带段和入口的可执行 ELF
  → execve 读取 ELF
  → 映射 PT_LOAD
  → 动态加载器完成库与重定位
  → 跳转 _start
  → 进入 main
```

理解了这条链，链接错误、动态库缺失、文件大小异常、程序入口问题和调试信息缺失，都能落到 ELF 的具体结构中分析。